myMNIST: Benchmark of PETNN, KAN, and Classical Deep Learning Models for Burmese Handwritten Digit Recognition

Presenter: Ye Kyaw Thu
Language Understanding Lab., Myanmar
NECTEC, Thailand

Email: ykt.nlp.ai@gmail.com

21 Mar 2026
@ICNLP 2026,
Xi'an, China

ICNLP 2026

2026 8th International Conference on NATURAL LANGUAGE PROCESSING

MARCH 20 - 22, 2026 • XI'AN, CHINA

Co-Sponsored by

IEEE

Hosted by

西安邮电大学 通信与信息工程学院

Patrons

华南师范大学 · 南京大学 · 中国科学院信息工程研究所 · 香港中文大学（深圳） The Chinese University of Hong Kong, Shenzhen · 北京理工大学

The Hong Kong Polytechnic University 香港理工大学 · 亚太認知智能學會 · electronics · CANCER INNOVATION · Discover Data

Outlines

Motivation

BHDD Dataset

Experimental Setting

Results & Discussion

Conclusion

arXiv > cs > arXiv:2603.18597

Search...

Help | Adva

Computer Science > Computer Vision and Pattern Recognition

[Submitted on 19 Mar 2026]

myMNIST: Benchmark of PETNN, KAN, and Classical Deep Learning Models for Burmese Handwritten Digit Recognition

Ye Kyaw Thu, Thazin Myint Oo, Thepchai Supnithi

We present the first systematic benchmark on myMNIST (formerly BHDD), a publicly available Burmese handwritten digit dataset important for Myanmar NLP/AI research. We evaluate eleven architectures spanning classical deep learning models (Multi-Layer Perceptron, Convolutional Neural Network, Long Short-Term Memory, Gated Recurrent Unit, Transformer), recent alternatives (FastKAN, EfficientKAN), an energy-based model (JEM), and physics-inspired PETNN variants (Sigmoid, GELU, SiLU). Using Precision, Recall, F1-Score, and Accuracy as evaluation metrics, our results show that the CNN remains a strong baseline, achieving the best overall scores (F1 = 0.9959, Accuracy = 0.9970). The PETNN (GELU) model closely follows (F1 = 0.9955, Accuracy = 0.9966), outperforming LSTM, GRU, Transformer, and KAN variants. JEM, representing energy-based modeling, performs competitively (F1 = 0.9944, Accuracy = 0.9958). KAN-based models (FastKAN, EfficientKAN) trail the top performers but provide a meaningful alternative baseline (Accuracy ~0.992). These findings (i) establish reproducible baselines for myMNIST across diverse modeling paradigms, (ii) highlight PETNN's strong performance relative to classical and Transformer-based models, and (iii) quantify the gap between energy-inspired PETNNs and a true energy-based model (JEM). We release this benchmark to facilitate future research on Myanmar digit recognition and to encourage broader evaluation of emerging architectures on regional scripts.

Preprint Link: https://arxiv.org/abs/2603.18597

Motivation

- **The Gap:** High-resource benchmarks (like original MNIST) don't capture the morphological complexity of regional scripts like Myanmar.
- **The Script:** Burmese digits involve looped strokes and curved ligatures, leading to unique visual confusions.
- **Goal:** Establish the first systematic, "apples-to-apples" benchmark on the *BHDD* dataset across diverse modeling paradigms.

Motivation

- **Previous Work:** Prior studies achieved 98.7% accuracy using ConvNets on BHDD (Thura Aung, 2024).
- Recent models like Kolmogorov-Arnold Networks (KAN) and Energy Transition Neural Networks (PETNN) haven't been tested on this script until now.

BHDD Dataset

- Dataset prepared by **Expa.AI Research Team**
- GitHub Link: https://github.com/baseresearch/BHDD

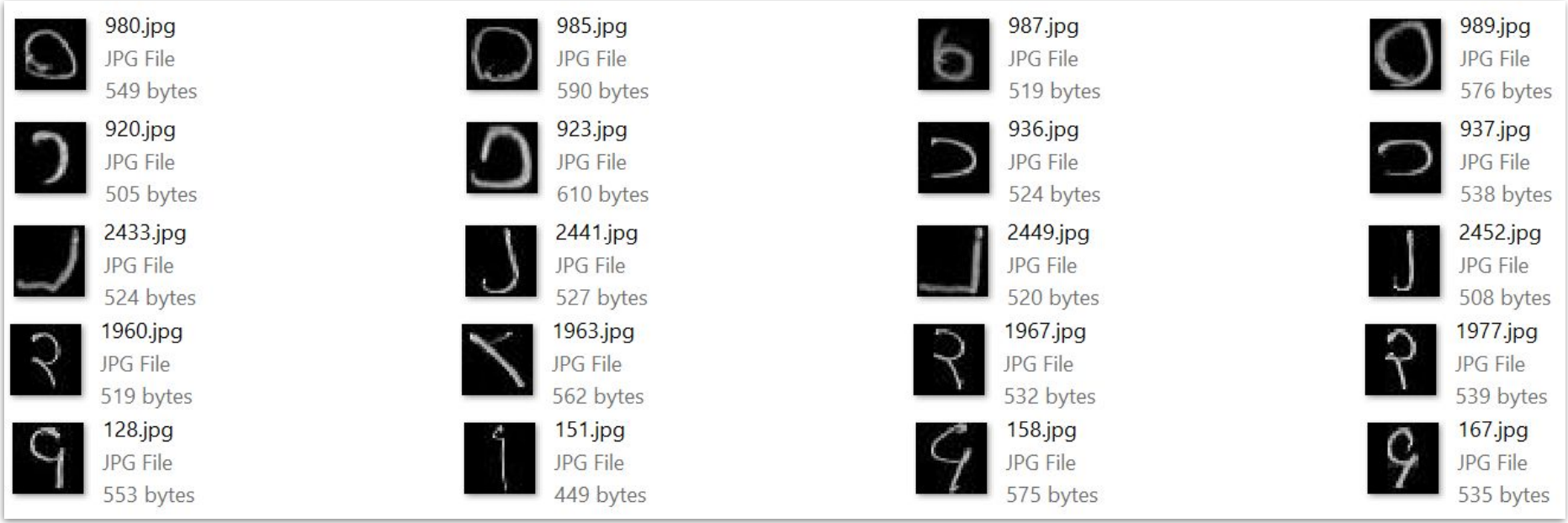


Fig. Some examples of test data (for 0 to 4)

BHDD Dataset

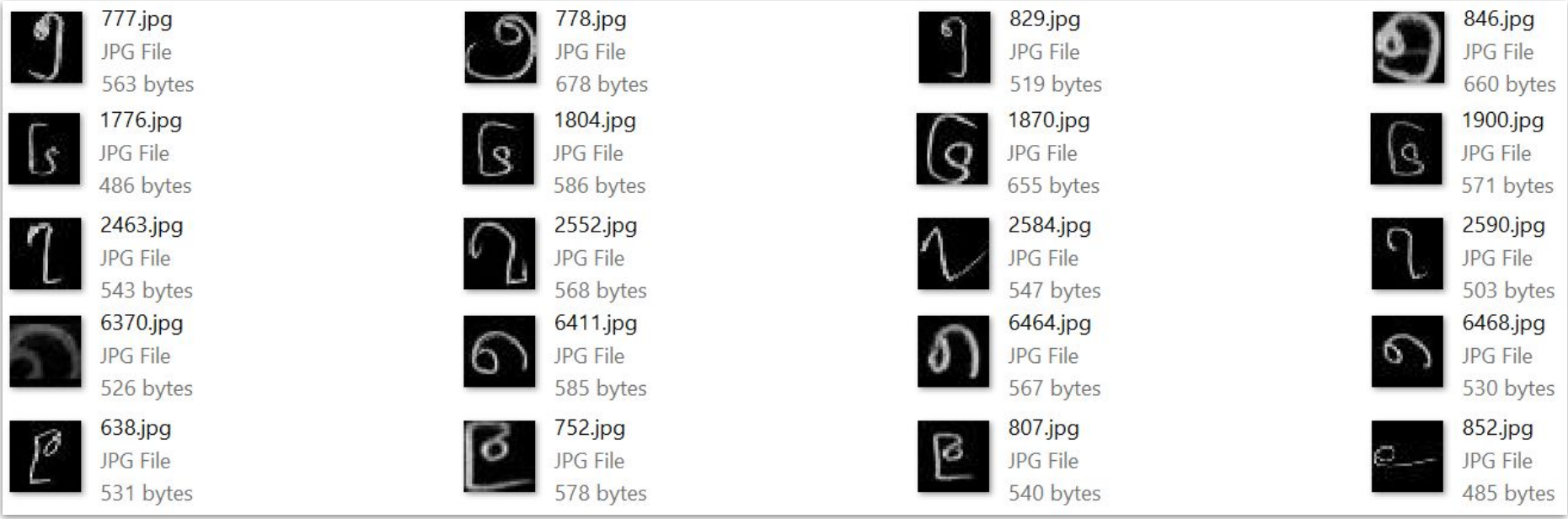


Fig. Some examples of test data (for 5 to 9)

Experimental Setting: Methodology

- **Classical Deep Learning:** MLP, CNN (VGG-style), LSTM, GRU, Transformer.
- **KAN Variants:** FastKAN, EfficientKAN.
- **Energy-Centric Models:** Joint Energy Model (JEM) and three PETNN variants (Sigmoid, GELU, SiLU).

Experimental Setting: Physics-inspired & Energy-based Models

- **PETNN:** Uses novel gated dynamics inspired by quantum state transitions; we re-implemented this to test three activation functions (Z. Wu, et. al, 2025).

$$T_t = R_t \cdot \sigma(T_{t-1} + Z_t) - 1 \quad \text{(Transition gate)} \tag{18}$$

$$m_t = \mathbb{I}(T_t \leq 0) \quad \text{(Transition mask)} \tag{19}$$

$$C_t = (1 - m_t)C_{t-1} + m_t I_t + Z_c \quad \text{(Memory cell)} \tag{20}$$

$$\mathbf{h}_t = \text{LayerNorm}(\sigma((1 - Z_w)\mathbf{h}_{t-1} + Z_w \mathbf{h} + \text{res}(\mathbf{x}_t))) \tag{21}$$

Experimental Setting: Physics-inspired & Energy-based Models

- **JEM:** Unifies discriminative and generative learning through an energy-based formulation.

$$p_\theta(\mathbf{x}, y) = \frac{e^{-E_\theta(\mathbf{x}, y)}}{Z(\theta)}, \quad E_\theta(\mathbf{x}, y) = -f_\theta(\mathbf{x})[y] \tag{15}$$

- Training combines cross-entropy with contrastive divergence using Langevin dynamics:

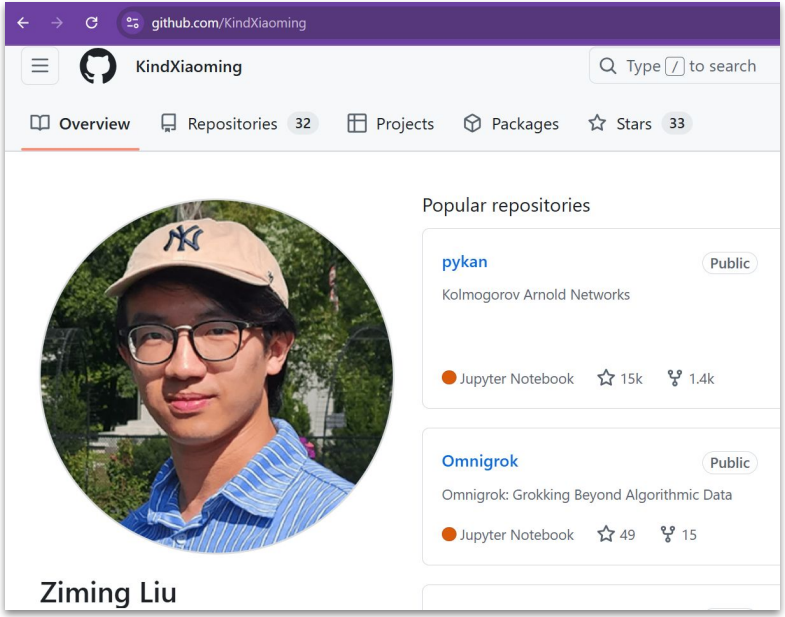
Experimental Setting: Kolmogorov-Arnold Networks (KAN)

KAN: Kolmogorov–Arnold Networks

Ziming Liu[1,4*] **Yixuan Wang**[2] **Sachin Vaidya**[1] **Fabian Ruehle**[3,4]
James Halverson[3,4] **Marin Soljačić**[1,4] **Thomas Y. Hou**[2] **Max Tegmark**[1,4]
[1] Massachusetts Institute of Technology
[2] California Institute of Technology
[3] Northeastern University
[4] The NSF Institute for Artificial Intelligence and Fundamental Interactions

Abstract

Inspired by the Kolmogorov-Arnold representation theorem, we propose Kolmogorov-Arnold Networks (KANs) as promising alternatives to Multi-Layer Perceptrons (MLPs). While MLPs have *fixed* activation functions on *nodes* ("neurons"), KANs have *learnable* activation functions on *edges* ("weights"). KANs have no linear weights at all – every weight parameter is replaced by a univariate function parametrized as a spline. We show that this seemingly simple change makes KANs outperform MLPs in terms of accuracy and interpretability, on small-scale AI + Science tasks. For accuracy, smaller KANs can achieve comparable or better accuracy than larger MLPs in function fitting tasks. Theoretically and empirically, KANs possess faster neural scaling laws than MLPs. For interpretability, KANs can be intuitively visualized and can easily interact with human users. Through two examples in mathematics and physics, KANs are shown to be useful "collabo-



Z. Liu, et al. 2024,

https://github.com/KindXiaoming

Experimental Setting: Kolmogorov-Arnold Networks (KAN)

- **The Shift:** Replaces fixed linear weights with learnable spline-parametrized univariate functions on the edges.
- **Feature:** Spline activations adapt during training, providing smooth gradients and high interpretability.

Experimental Setting: MLP vs KAN (for shallow model)

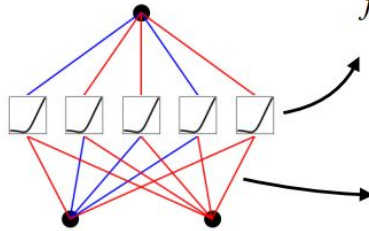
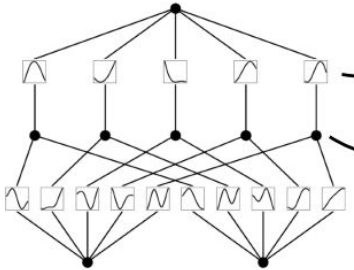
Model	**Multi-Layer Perceptron (MLP)**	**Kolmogorov-Arnold Network (KAN)**
Theorem	**Universal Approximation Theorem**	**Kolmogorov-Arnold Representation Theorem**
Formula (Shallow)	$f(\mathbf{x}) \approx \sum_{i=1}^{N(\epsilon)} a_i \sigma(\mathbf{w}_i \cdot \mathbf{x} + b_i)$	$f(\mathbf{x}) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right)$
Model (Shallow)	**(a)**  *fixed* activation functions on *nodes* *learnable* weights on *edges*	**(b)**  *learnable* activation functions on *edges* sum operation on *nodes*

Fig. Multilayer Perceptron (MLP) vs Kolmogorov-Arnold Network (KAN)

Experimental Setting: MLP vs KAN (for deep model)

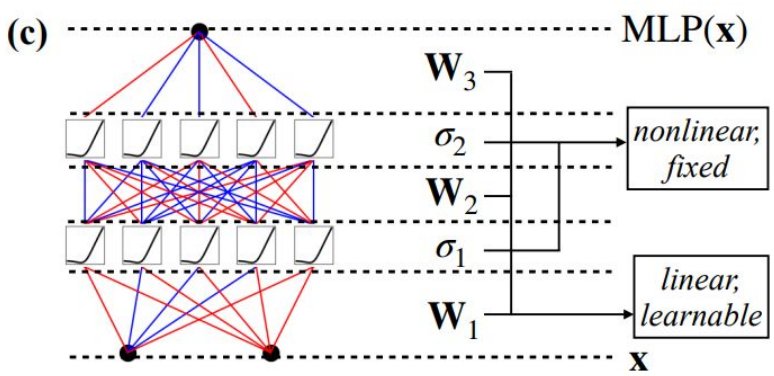
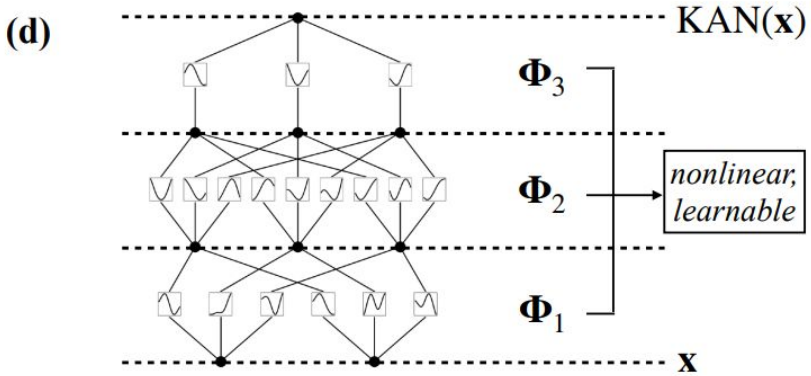
Formula (Deep)	$\mathrm{MLP}(\mathbf{x}) = (\mathbf{W}_3 \circ \sigma_2 \circ \mathbf{W}_2 \circ \sigma_1 \circ \mathbf{W}_1)(\mathbf{x})$	$\mathrm{KAN}(\mathbf{x}) = (\mathbf{\Phi}_3 \circ \mathbf{\Phi}_2 \circ \mathbf{\Phi}_1)(\mathbf{x})$
Model (Deep)		

Fig. Multilayer Perceptron (MLP) vs Kolmogorov-Arnold Network (KAN)

Experimental Setting

- **Hardware:** Single NVIDIA GeForce RTX 3090 Ti GPU.
- **Framework:** PyTorch with shared optimization (AdamW, OneCycleLR).
- **Protocol:** Unified training configuration for fair comparison; fixed seeds for reproducibility.

Results

Model	Precision	Recall	F1-Score	Accuracy
MLP	0.9810	0.9895	0.9852	0.9907
CNN	0.9955	0.9963	**0.9959**	**0.9970**
LSTM	0.9907	0.9942	0.9924	0.9951
GRU	0.9886	0.9934	0.9910	0.9937
Transformer	0.9898	0.9944	0.9921	0.9946
JEM	0.9931	0.9957	0.9944	0.9958
FastKAN	0.9844	0.9914	0.9879	0.9922
EfficientKAN	0.9841	0.9898	0.9869	0.9918
PETNN (Sigmoid)	0.9897	0.9940	0.9918	0.9943
PETNN (GELU)	0.9947	0.9963	0.9955	0.9966
PETNN (SiLU)	0.9944	0.9961	0.9952	0.9964

TABLE III: Performance comparison of various models on the BHDD dataset.

Results

- CNN holds the top score with 99.70% Accuracy (0.9959 F1).
- PETNN (GELU) is a very close second (99.66% Accuracy).
- KAN variants performed respectably (~99.2%) but lagged behind PETNN and JEM.

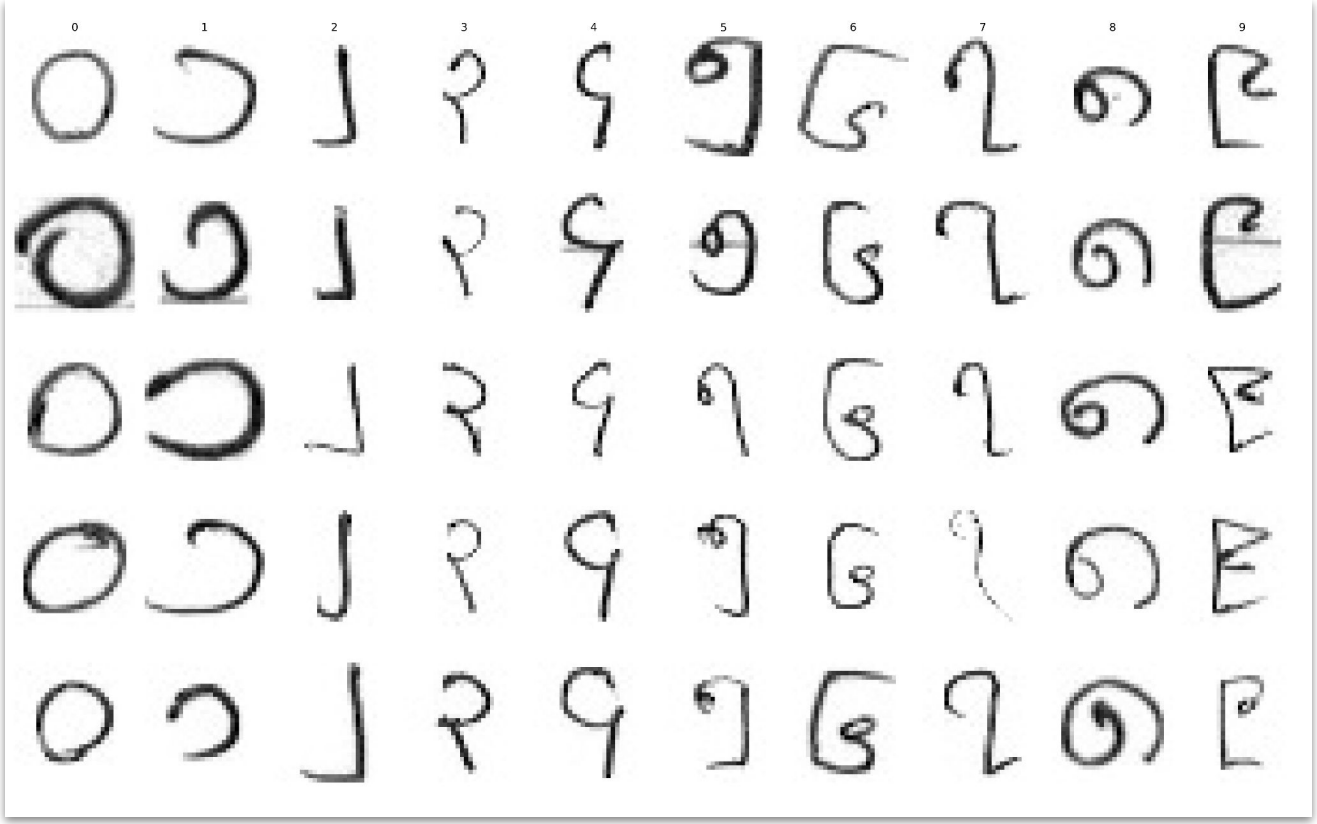
Discussion

- **PETNN Efficiency:** Remarkably fast; only ~20 seconds per epoch compared to 110s for Transformer.
- **GPU Usage:** PETNN used ~1.8GB VRAM, while JEM required ~4.0GB.
- PETNN performance is highly sensitive to the gating activation function.
- GELU and SiLU significantly outperformed Sigmoid.

Error Analysis

- Major Error: 1 (ɔ) ↔ 0 (o).
- Cause: 1 is an open-circular stroke; 0 is closed. Fast handwriting often closes the gap in 1 or leaves a gap in 0.
- This is a visual ambiguity problem, not a model deficiency, as it affects all 11 architectures.

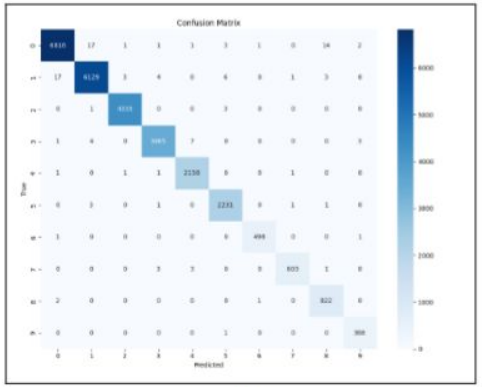
Error Analysis



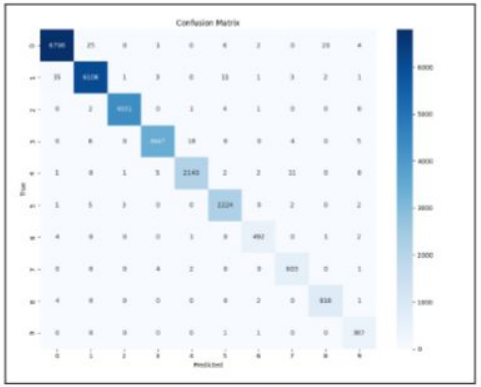
Error Analysis

- 3 () → 4 () confusion in KAN models.
- 3 opens left; 4 opens right.
- KAN architectures may struggle with fine-grained feature extraction of stroke direction and closure position.

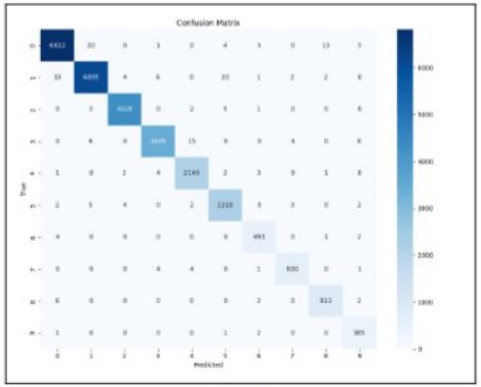
Error Analysis



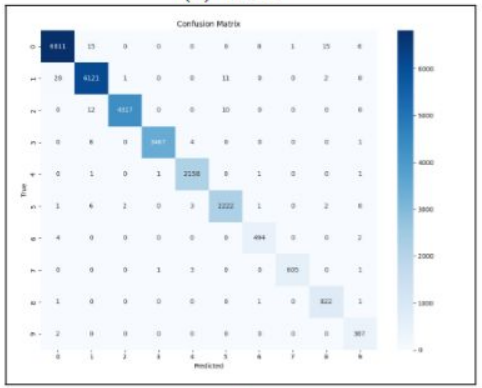
(a) JEM.



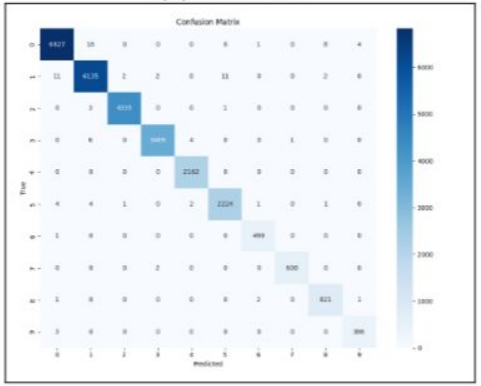
(b) fastKAN.



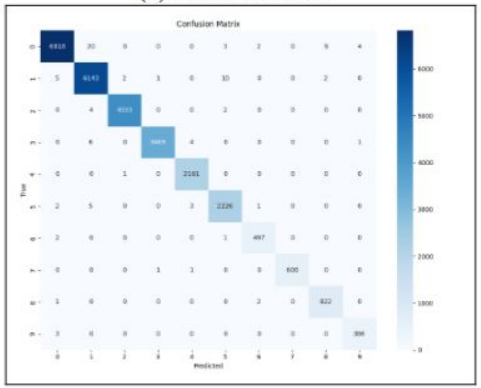
(c) efficientKAN.



(d) PETNN (Sigmoid).



(e) PETNN (GELU).



(f) PETNN (SiLU).

Conclusion

- Established first systematic benchmark for Burmese handwritten digits.
- CNN remains the strongest baseline, but PETNN (GELU) is a highly efficient alternative.
- Identified fundamental script-specific visual ambiguities that even modern models share.
- We plan to share our codes: https://github.com/ye-kyaw-thu/myMNIST-benchmark

References

1. Quick Intro to KAN: https://kindxiaoming.github.io/pykan/intro.html
2. Original Implementation: https://github.com/KindXiaoming/pykan
3. Efficient-KAN: https://github.com/Blealtan/efficient-kan
4. Fast-KAN: https://github.com/ZiyaoLi/fast-kan
5. FasterKAN: https://github.com/AthanasiosDelis/faster-kan
6. benchmarking-KAN: https://github.com/eleonorapoeta/benchmarking-KAN
7. Very Slow Information: https://github.com/ale93111/pykan_mnist
8. Ziming Liu Homepage: https://kindxiaoming.github.io/
9. MNIST and PyKan: https://www.kaggle.com/code/minsithu/mnist-and-pykan
10. Dissecting Kolmogorov-Arnold Network: https://feicheung2016.medium.com/dissecting-kolmogorov-arnold-network-f1bee719d949
11. "KAN: Kolmogorov-Arnold Networks" by Ziming Liu (about 58 min): https://www.youtube.com/watch?v=uupyXjSnZiA

References

12. BHDD Dataset: https://github.com/baseresearch/BHDD
13. Joint Energy Model (JEM), Grathwohl et al., 2019.
14. KAN: Kolmogorov-Arnold Networks, Liu et al., 2024.
15. PETNN, Wu et al., 2025.

Thank you!
Q&A

Ye Kyaw Thu
LU Lab., Myanmar
NECTEC, Thailand

Email: ykt.nlp.ai@gmail.com